

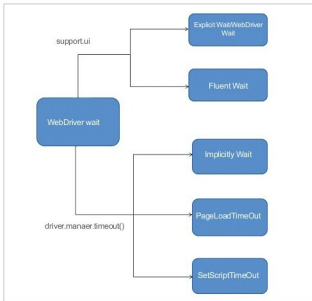


Figure 1: Types of waits

Declaration:

```
driver.manage().timeouts().pageLoadTimeout(50, TimeUnit.SECONDS);
```

When to use: Whenever you need to ensure a page is loaded in the optimum time before proceeding to perform an operation on WebElements.

- **SetScriptTimeout:** This wait is applicable only for JavascriptExecutor, which provides us two methods for script execution — `executeScript` and `executeAsyncScript`.

The `executeScript` method will block the execution of the next line of code till the execution of this method is completed, and then it moves to the next line of code. The `executeAsyncScript` method does not block the execution of the next line of code till the execution of this method is completed.

Since the `executeAsyncScript` method doesn't block the execution of the next line of code, we use `driver.manage().timeouts().setScriptTimeout(5,SECONDS)`; to specify the amount of time for an asynchronous script to finish the execution before throwing an error.

Declaration:

```
driver.manage().timeouts().setScriptTimeout(5, TimeUnit.SECONDS);
```

When to use: Used to specify the amount of time for an asynchronous script to finish execution, or else it will take its own time to execute and may allow a bug to slip away.

- **FluentWait:** Each `FluentWait` instance defines the maximum amount of time to wait for a condition, as well as the frequency with which to check the

condition. The user may configure the wait to ignore specific types of exceptions while waiting, such as `NoSuchElementException` when searching for an element on the page, and to ignore the exception to continue with script execution in case the element is not found.

Declaration:

```
Wait<WebDriver> wait = new FluentWait<WebDriver>(driver)
    .withTimeout(30, TimeUnit.SECONDS).pollingEvery(5,
    TimeUnit.SECONDS).ignoring(NoSuchElementException.class);
```

```
WebElement clickSeleniumLink = wait.until(new Function<WebDriver,
    WebElement>() {public WebElement
    apply(WebDriver driver) {
        return driver.findElement(By.xpath("//*[@id='login-
        button']"));
    }
    });
```

When to use: Used when the element takes a different loading time to appear each time we run the test.

- **Explicit wait:** This allows the user to direct the driver to wait for certain conditions to occur before proceeding further in the code. `WebDriverWait` by default calls the `ExpectedCondition` every 500 milliseconds until it returns successfully.

Declaration:

```
WebDriverWait wait = new WebDriverWait(driver, 10);
wait.until(ExpectedConditions.elementToBeClickable(By.
    xpath("//*[@id='login-button']"))).click();
```

When to use: When the element takes a long time to load and also when there is a need to check the property of an element (e.g., element is clickable).

Effective use of waits in a real world

Using a combination of `pageLoadTimeout` + `SetScriptTimeout` and `ExplicitWait` in your framework ensures that the page is loaded at the optimal time, and that all asynchronous scripts are executed in a stipulated time. `ExplicitWait` helps to check the various properties of different elements based on the type of operation that needs to be performed. If there are any failures, it's easy to find the root cause for them. Using this combination of waits in your framework can deliver the most robust and accurate results. **END**

Reference

<http://www.seleniumhq.org/docs/>

By: Athmeeya N.R.

The author works as a Selenium automation developer at DellEmc (EMC2) and has six years of experience in automation testing.